PROJECT 1 — GOLDEN CRUST

System Analysis, Design, Development, Testing & Deployment

Project type: Web/Application Development · shared group portfolio project **Brand:** Golden Crust (original code and references may say BreadStore) **Stack:** Python, FastAPI, SQLite, HTML, CSS, JavaScript and Bootstrap **Live admin site:** https://golden-crust-demo.onrender.com/ **Customer storefront:** https://golden-crust-demo.onrender.com/shop **Source repository:** https://github.com/Frostner/BreadStore-Inventory-System **Documentation evidence:** Accompanying BreadStore source-code archive and its docs/ folder; manual live-ordering confirmation supplied by student.

1. Executive overview

Golden Crust is a bakery management prototype consisting of an administrator dashboard and a customer-facing shop. Staff manage products, inventory, point-of-sale transactions, orders and reports. Customers register, browse bread products, place orders for pickup and check order history. Both views use the same FastAPI backend and SQLite relational database. The site is a public **portfolio demonstration**, not a production commerce platform.

2. Problem statement and existing/proposed system

Scenario: A small bakery relying on handwritten stock counts and sales records can have delayed inventory updates, missing records, difficulty checking daily performance and ordering confusion. This is a proposed problem scenario, **not a claim that a specific bakery was surveyed**.

Proposed solution: A centralized inventory and sales database accessible via an admin browser and a customer ordering interface. The system rejects invalid stock requests, tracks product movements, reserves items when pickup orders are placed, and records completed transactions in reports.

3. Objectives

- Provide authenticated admin inventory and product maintenance functions.
- Maintain product quantities and a stock-movement history.
- Provide customer registration, storefront browsing, cart and pickup ordering.
- Let administrators process orders through a defined status lifecycle.
- Save POS sales and completed pickup transactions in consistent records.
- Offer inventory reporting, CSV export and reorder suggestions.
- Demonstrate client/server architecture, SQL data relationships, validation and deployment.

4. Scope and functional requirements

ID	Requirement	Implemented evidence
FR01	Administrator login/logout	/api/auth/login, /api/auth/logout
FR02	Product create/read/update/archive	Product APIs and admin inventory panel
FR03	Product search, filtering and validation	Server routes and admin interface
FR04	Stock adjustments and audit log	stock_movements and adjustment endpoints
FR05	Point-of-sale checkout	Sales and sale-item records
FR06	Customer registration, authentication and product browser	/shop and /api/shop/*
FR07	Submit pickup orders with stock reservation	Customer order routes and transactions
FR08	Admin order status transitions	Pending → Preparing → Ready → Completed, or Cancelled
FR09	Reports, trends and CSV exports	Dashboard/report/export endpoints
FR10	Friendly failures for insufficient stock and invalid inputs	Pydantic rules and SQL constraints

Out of scope: Delivery logistics, online card charging, email/SMS notification, formal production payment verification and guaranteed online persistence on the Render Free plan.

5. Nonfunctional requirements

- **Usability:** responsive layouts for computer and mobile; error messages visible in the interface.
- **Data integrity:** atomic SQLite transactions for checkout; stock may not be negative.
- **Security:** salted PBKDF2 password hashes, HttpOnly sessions, validated input, parameterized SQL and required request headers. These are safeguards, not a claim of an independently audited security system.
- **Maintainability:** separate backend modules, frontend assets, schema and tests.
- **Availability:** live public Render URL; cold starts and resettable data on free hosting are known limitations.

6. Users and use cases

Customer: register, sign in, browse products, add to basket, submit pickup order, view order status, cancel own order while Pending. **Administrator:** sign in, manage inventory, monitor stock, record in-store sales, view customer orders, move orders through statuses, review reports and export records.

7. Architecture and data flow

Customer / Admin Browser → FastAPI routes and business validation → SQLite database → JSON response → Browser display.

- app.py defines admin and shared API routes and serves static frontend content.
- customer_routes.py defines storefront, customer authentication, pickup order and fulfillment routes.
- database.py initializes tables, hashing and sample records.
- static/index.html, static/shop.html, associated JavaScript and CSS make up the interfaces.
- The SQLite file is normally stored at data/breadstore.db and should not be pushed publicly with real customer data.

8. Database design and relationships

Table	What it stores
users, sessions	Admin account and session-token hashes
products	SKU, category, unit cost/price in centavos, quantity, stock threshold, archive state
stock_movements	Product, quantity delta, reason, note, actor and timestamp
sales, sale_items	Completed receipts and price snapshots
customers, customer_sessions	Customer accounts and separate authentication sessions
customer_orders, customer_order_items	Pickup orders, status, buyer, product quantities and locked-in price snapshots

Key relationships: one customer can place multiple orders; each order contains multiple order items; products appear in order and sale items; completed customer orders can link to a sale. See original schema.sql and docs/CUSTOMER_ORDERING.md for the SQL definitions and ERD.

9. Core workflows

A. Customer pickup — Customer signs in → browses /shop → adds items → submits pickup order → server validates requested stock and current prices → SQLite transaction saves order and reserves stock → status Pending. Staff later moves it to Preparing → Ready → Completed after simulated pickup/payment. Completion adds a sale and **does not deduct stock twice**.

B. Cancellation — Customer can cancel a Pending order; staff can cancel before completion. Cancellation restores the reserved quantity once and saves a stock movement.

C. In-store POS — Admin selects products → checks availability → transaction records sales, sale_items, stock updates and movement entries together. If stock is insufficient, transaction is rejected.

10. Testing and verification

The provided October 20 source documentation reports **11 automated integration tests** across tests/test_api.py and tests/test_customers.py. They cover auth, product validation, inventory, checkout rollback, customer registration/ordering and order fulfillment. **Those test results come from the bundled source's documentation; they have not been rerun as part of this portfolio update.**

Student-confirmed manual evidence: Successfully deployed on Render (Oct 28, 2026), logged into the live admin dashboard (HTTP 200 observed in Render screenshot), and placed a customer order online and confirmed the flow worked, according to the student. Other test cases are documented in the separate testing matrix as **recommended to verify**, not claimed passed.

Test	Expected outcome	Evidence state
Admin login	Dashboard loads	Live Render log shows login success
Customer order placement	Order accepted and reaches admin	Confirmed by student in chat
Invalid admin login	Rejected without session	Provided test suite; student rerun recommended
Duplicate product SKU	Validation error	Provided test suite; student rerun recommended
Overselling	Rejected without partial stock deduction	Provided test suite; student rerun recommended
Customer cancellation	Stock reserved earlier is restored	Provided customer-order docs; student rerun recommended
Picked-up order completion	Sale created; no double stock deduction	Provided customer-order docs; student rerun recommended

Test	Expected outcome	Evidence state
Resume after local restart	SQLite persists locally	Student manual verification recommended

11. Installation and configuration

Windows/VS Code instructions from the supplied source archive:

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app:app --reload
```

Open http://127.0.0.1:8000/ for the admin view and /shop for customers. Copy .env.example to .env, choose a unique strong admin password and keep that file out of GitHub. In the actual student setup, the provided START_BREADSTORE.bat was used for local startup. Follow the README if command flags differ in the current repository.

12. Deployment evidence and configuration

Live public site runs on **Render Web Service, Python 3, Free $0/month**. Build command: pip install -r requirements.txt. Start command: uvicorn app:app --host 0.0.0.0 --port $PORT. In Render, ADMIN_PASSWORD is stored privately and should never be in documentation. COOKIE_SECURE=true, SEED_DEMO=true and a supported Python runtime were configured at deployment.

Observed evidence: Render showed Deploy succeeded | Live on Oct 28, 2026. Web requests for /, JavaScript, images and successful admin authentication had HTTP 200 responses. The customer ordered successfully online afterward.

Free-host limitation: Render's ephemeral filesystem may discard newly recorded SQLite customer/order data on restart, sleep or redeployment. Sample records can be reseeded. The local copy remains the reliable demo for persistence; the public deployment is a visual and workflow demonstration.

13. Problems and resolutions

- **Local runtime startup:** Early FastAPI compatibility issue linked to unsupported Query(enum=...) in an older code snapshot. The student verified no matching enum= call in the pushed GitHub app on Oct 28.
- **Render startup:** Replaced the default WSGI-style command with the correct FastAPI/Uvicorn command.
- **Cost configuration:** Changed the selected instance from paid to Free before deploying.
- **Test data persistence:** Documented the cloud storage limitation and retained the local database for school demonstration.

14. Limitations and future improvements

Persistent database hosting (e.g., a suitable managed database or paid persistent storage), role-based accounts, production payment integration, pickup notifications, stronger privacy disclosures, backups and retention policies, and independent security/performance testing. Reorder advice is rule-based rather than a trained AI model.

15. Evidence and attribution

This is described in the student's website as a **group project**. Each member should only claim work they actually performed; shared repository links do not prove personal implementation. The live URL, code archive, original screenshots, customer ordering guide and test files are attached as evidence. No production customer data should be used.